

Q1A:

I implemented a Quicksort algorithm for sorting items stored in Array Lists every day of the week during this part of my task. For each day, random items of between 1000 and 500000 were generated with the intention that they would be stored in arrays lists as well as sorted by Quicksort algorithm.

```
Java — -zsh — 80x24
sarahalrefaie@Sarahs-Laptop desktop % cd java
sarahalrefaie@Sarahs-Laptop java % javac SortingItems.java

sarahalrefaie@Sarahs-Laptop java % java SortingItems

Unsorted lists:
4665 3832 4982 4746 1374 2174 1898 1466 1917 3317
1310 1773 3181 1179 4708 3296 3355 1656 4931 2595
4624 4680 2338 3267 4444 1267 3824 4898 3648 4057
3953 3234 3954 4906 1564 3273 4035 4486 1246 2115
2761 1726 4820 2391 1516 2060 2723 1360 3487 1015
1224 3712 2836 3837 2629 3142 2672 3439 3371 3990
4728 2383 3101 3387 4648 3511 4198 1884 2112 2638

Sorted lists:
1374 1466 1898 1917 2174 3317 3832 4665 4746 4982
1172 1179 1310 1369 1656 1773 1775 2484 2595 2856
1177 1267 1300 1379 1388 1456 1491 1537 1996 2031
1161 1246 1275 1284 1564 1732 1840 2048 2115 2214
1015 1142 1252 1360 1494 1515 1516 1542 1657 1659
1058 1096 1158 1224 1269 1298 1851 1925 1991 2053
1058 1060 1092 1206 1222 1237 1393 1415 1436 1575
sarahalrefaie@Sarahs-Laptop java %
sarahalrefaie@Sarahs-Laptop java %
```

Pseudocode for Q1A

```
1  QUICKSORT(array, low, high)
2  |   if low < high
3  |   |   pivot_index = PARTITION(array, low, high)
4  |   |   QUICKSORT(array, low, pivot_index - 1)
5  |   |   QUICKSORT(array, pivot_index + 1, high)
6  |
7  PARTITION(array, low, high)
8  |   pivot = array[high]
9  |   i = low - 1
10 |   for j = low to high - 1
11 |   |   if array[j] <= pivot
12 |   |   |   i++
13 |   |   |   SWAP(array[i], array[j])
14 |   |   SWAP(array[i + 1], array[high])
15 |   return i + 1
```

Q1B

```
[sarahalrefaie@Sarahs-Laptop java % java EfficiencyTesting
List size: 1000
Random list time: 1 ms
Sorted list time: 10 ms
Reverse sorted list time: 3 ms

List size: 5000
Random list time: 0 ms
Sorted list time: 35 ms
Reverse sorted list time: 23 ms

List size: 10000
Random list time: 1 ms
Sorted list time: 162 ms
Reverse sorted list time: 92 ms

List size: 50000
Random list time: 5 ms
Sorted list time: 4071 ms
Reverse sorted list time: 2696 ms

List size: 75000
Random list time: 8 ms
Sorted list time: 9336 ms
Reverse sorted list time: 6067 ms

List size: 100000
Random list time: 13 ms
Sorted list time: 16580 ms
Reverse sorted list time: 10289 ms

List size: 500000
Random list time: 66 ms
Sorted list time: 474047 ms
Reverse sorted list time: 306332 ms

sarahalrefaie@Sarahs-Laptop java %
```

Q1C

I encountered problems with large lists when trying to use Quicksort for Q1A in the beginning. They often caused stack overflow errors. Q1B efficiency tests have shown that Quicksort is not capable of handling multiple or reversed sorted lists. To solve these problems, I decided to create a hybrid sorting algorithm for Q1C.

My hybrid algorithm combines Quicksort with insertion sort. It starts with Quicksort, but when the list gets small enough, it switches to insertion sort. This way, it uses Quicksort's strength for big lists and insertion sort's strength for small ones. I tested the hybrid algorithm using Q1-B's efficiency testing framework, which helped me find and fix errors and make improvements. I made the algorithm much faster by adjusting the cut-off value and optimising the insert sort part.

The final hybrid sorting algorithm works almost as well as Quicksort for random lists but does a lot better with sorted and reverse-sorted lists. It combines the best of both worlds, which makes it easy to sort lists for large and small items.

To increase efficiency, I might try different cut-off values and pivot selection strategies so that the algorithm can be improved.

```
[sarahalrefaie@Sarahs-Laptop java % java TestHybridSorting
Testing Hybrid Sorting Algorithm:
List size: 1000
Random list time: 1 ms
Sorted list time: 15 ms
Reverse sorted list time: 3 ms

List size: 5000
Random list time: 3 ms
Sorted list time: 101 ms
Reverse sorted list time: 55 ms

List size: 10000
Random list time: 2 ms
Sorted list time: 295 ms
Reverse sorted list time: 191 ms List size: 50000
Random list time: 9 ms
Sorted list time: 7089 ms
Reverse sorted list time: 4141 ms

List size: 75000
Random list time: 15 ms
Sorted list time: 15892 ms
Reverse sorted list time: 9270 ms

List size: 100000
Random list time: 14 ms
Sorted list time: 27913 ms
Reverse sorted list time: 16521 ms

List size: 500000
Random list time: 89 ms
Sorted list time: 826166 ms
Reverse sorted list time: 592352 ms

sarahalrefaie@Sarahs-Laptop java %
```